Devops Assessment Cisco

Deployment Guide

1. Introduction

This project is a full-stack application consisting of:

- **Frontend**: React (TypeScript)
- **Backend**: Rust (API service)

The objective is to design a **production-ready deployment strategy**, including containerization, CI/CD pipeline, and scalability considerations.

2. Architecture Overview

Chosen Stack

- **Containerization**: Docker
- **Container Registry**: AWS ECR
- **Orchestration**: AWS ECS (Fargate) *(or Kubernetes optional)*
- **Load Balancer**: AWS Application Load Balancer (ALB)
- **CI/CD**: GitHub Actions / Jenkins
- **Infrastructure as Code**: Terraform

3. High-Level Architecture

User → ALB → ECS Services
— Frontend (React + Nginx)
— Backend (Rust API)

- ALB routes:
  - / → frontend
  - /api → backend

4. Dockerization Strategy

Backend (Rust)

Multi-stage build to reduce size:

```
FROM rust:1.75 as builder
WORKDIR /app
COPY . .
RUN cargo build --release

FROM debian:slim
COPY --from=builder /app/target/release/app /usr/local/bin/app
CMD ["app"]
```

Frontend (React)

```
FROM node:18 as builder
WORKDIR /app
COPY . .
RUN npm install && npm run build

FROM nginx:alpine
COPY --from=builder /app/build /usr/share/nginx/html
```

5. Deployment Flow

1. Developer pushes code to GitHub
2. CI/CD pipeline triggers
3. Build Docker images (frontend + backend)
4. Push images to AWS ECR
5. Update ECS service with new image
6. ALB routes traffic to updated containers

6. CI/CD Pseudocode

name: CI-CD Pipeline

```
on:
  push:
    branches: [main]

jobs:
  build-deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
```

```
    uses: actions/checkout@v3

  - name: Build Backend Image
    run: docker build -t backend ./api

  - name: Build Frontend Image
    run: docker build -t frontend ./app

  - name: Login to ECR
    run: aws ecr get-login-password | docker login ...

  - name: Push Images
    run: |
      docker tag backend <ECR_URI_BACKEND>
      docker push <ECR_URI_BACKEND>
      docker tag frontend <ECR_URI_FRONTEND>
      docker push <ECR_URI_FRONTEND>

  - name: Deploy to ECS
    run: aws ecs update-service --cluster my-cluster --service my-service --force-new-deployment
```

7. Infrastructure (Terraform)

- ECS Cluster
- Task Definitions (frontend + backend)
- ALB with routing rules
- IAM roles
- ECR repositories

8. Scalability & Reliability

- Auto Scaling (CPU / Memory based)
- Stateless containers
- Health checks via ALB
- Rolling deployments

9. Security Best Practices

- Use **IAM roles (IRSA)** instead of static credentials
- Store secrets in **AWS Secrets Manager**
- Enable HTTPS via **ACM**
- Run containers as **non-root user**

10. Observability

- Logs → CloudWatch / ELK
- Metrics → Prometheus + Grafana
- Alerts → Slack / Email

11. Improvements (Important for Interview)

- Blue-Green / Canary deployment (via ALB or ArgoCD)
- Use Kubernetes (EKS) for better control
- Add caching (Redis)

- Use CDN (CloudFront) for frontend
- Add SLO/SLI monitoring

12. Conclusion
The system is designed to be:
- Scalable
- Secure
- Highly available
- CI/CD automated

2. WWWT Project – Findings Report

1. Introduction
This project is a Python-based application with:
- app.py → core logic
- requirements.txt
- Dockerfile
- images/ → large dataset

2. Key Observations

Application Type
- Likely Flask-based image processing or serving app
- Heavy dependency on local image files

3. Findings

Docker Issues
- No multi-stage build → large image size
- No .dockerignore
- Runs as root user → security risk
- Base image not pinned → instability

Security Issues
- No authentication mechanism
- No input validation
- Dependencies not pinned → supply chain risk

Performance Issues
- Images bundled inside container → huge size
- Slow startup time
- No caching layer

Code Quality Issues
- Monolithic structure (single file app.py)
- No logging strategy
- No exception handling framework

DevOps Gaps

- No CI/CD pipeline
- No environment separation (dev/stage/prod)
- No monitoring or alerting

Recommendations

Docker Improvements

```
FROM python:3.10-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

RUN useradd -m appuser
USER appuser

CMD ["python", "app.py"]
```

Security Enhancements

- Add input validation
- Use environment variables
- Pin dependencies

Performance Improvements

- Move images to **AWS S3**
- Use **CloudFront CDN**
- Add caching (Redis)

DevOps Improvements

- Add CI/CD pipeline (GitHub Actions)
- Add logging (Fluent Bit / ELK)
- Add monitoring (Prometheus)

Suggested CI/CD Flow

Code Push → Build Docker → Scan Image → Push → Deploy

Conclusion

The application works but is **not production-ready** due to:

- Security gaps
- Performance inefficiencies
- Missing DevOps practices

With improvements, it can scale to production-grade systems.